BANORTE

GUÍA DE TRABAJO

GITLAB

PARA USO EXCLUSIVO DE

BANORTE SEGUROS Y PENSIONES

ELABORADO POR:

ARQUITECTURA TI

BANORTE

GITLAB

Contenido

1. Antes de iniciar 4
2. ¿Qué es GitLab? 6
3. Primeros pasos 7
4. Generar clave SSH 8
5. Estructura GitLab 11
6. Nuevo Proyecto 13
7. Clonar un repositorio 16
8. Wiki 17
9. Un vistazo a Wiki 18
10. Trabajando con ramas 21
11. Integración Continua (CI) 23
12. Push un nuevo Merge, Commit y Pipeline 24
 12.1. Merge Request 24
 12.2. Commit 28
 12.3. Pipeline 29
13. Herramientas avanzadas 32
14. Instalación de Git 33
15. Configuración de Git 33

BANORTE

GITLAB

Contenido

16. Iniciar un Proyecto 33
16.1. Trabajo diario 34
16.2. Modelo de ramificación 35
16.3. Revisa tu trabajo 36
16.4. Etiquetado 37
16.5. Revertir cambios 38
16.6. Sincronizar repositorio 39
16.7. Seguimiento y cambio de ruta 39
16.8. Commits temporales 40
16.9. Ignorar archivos 41
16.10. Documentación 41

Antes de iniciar

GitLab es una excelente manera de administrar repositorios de git en un servidor centralizado. GitLab brinda un control completo sobre sus repositorios o proyectos y le permite decidir si son públicos o privados de forma gratuita.

Características

- GitLab aloja sus proyectos de software (privados) de forma gratuita.
- GitLab es una plataforma para administrar repositorios de Git.
- GitLab ofrece repositorios públicos y privados gratuitos, seguimiento de problemas y wikis.
- GitLab es una capa de interfaz web fácil de usar sobre Git, que aumenta la velocidad de trabajo con Git.
- GitLab proporciona su propio sistema de Integración Continua (CI) para administrar los proyectos y proporciona una interfaz de usuario junto con otras características de GitLab.

Ventajas

- GitLab proporciona la versión GitLab Community Edition para que los usuarios localicen en qué servidores está presente su código.
- GitLab proporciona un número ilimitado de repositorios públicos y privados de forma gratuita.
- La sección Snippet puede compartir una pequeña cantidad de código de un proyecto, en lugar de compartir todo el proyecto.

Desventajas

- Al hacer push y extraer repositorios, no es tan rápido como GitHub.
- La interfaz de GitLab llevará tiempo al cambiar de una página a otra.

¿Qué es GitLab?

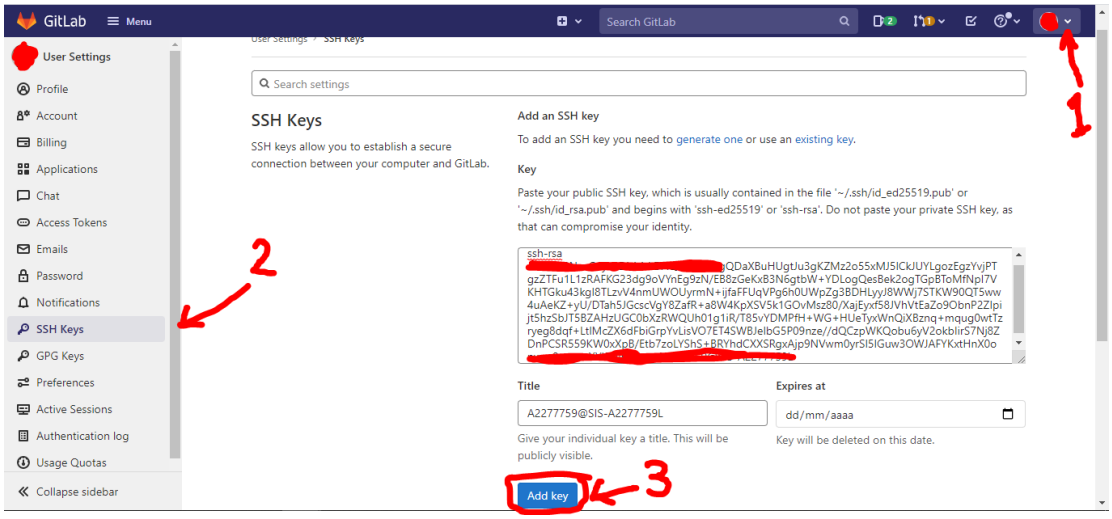
Es un servicio web de control de versiones y desarrollo de software colaborativo basado en Git. Además de gestor de repositorios, el servicio ofrece también alojamiento de wikis y un sistema de seguimiento de errores, todo bajo una Licencia de código abierto.

- Permite a las personas interactuar con uno o más repositorios de git mediante el uso de una interfaz web amigable.
- Amplía la funcionalidad del repositorio con herramientas dedicadas (que en realidad hacen la vida mucho más fácil).
- Permite trabajar en equipo integrando los pasos de desarrollo, pruebas y entrega, organizando y moderando el trabajo en equipo entre integrantes.
- Hay proyectos alternativos que ofrecen funcionalidades similares (GitHub, BitBucket, etc.…) pero hasta ahora GitLab parece ser la solución todo en uno más completa.

Primeros Pasos

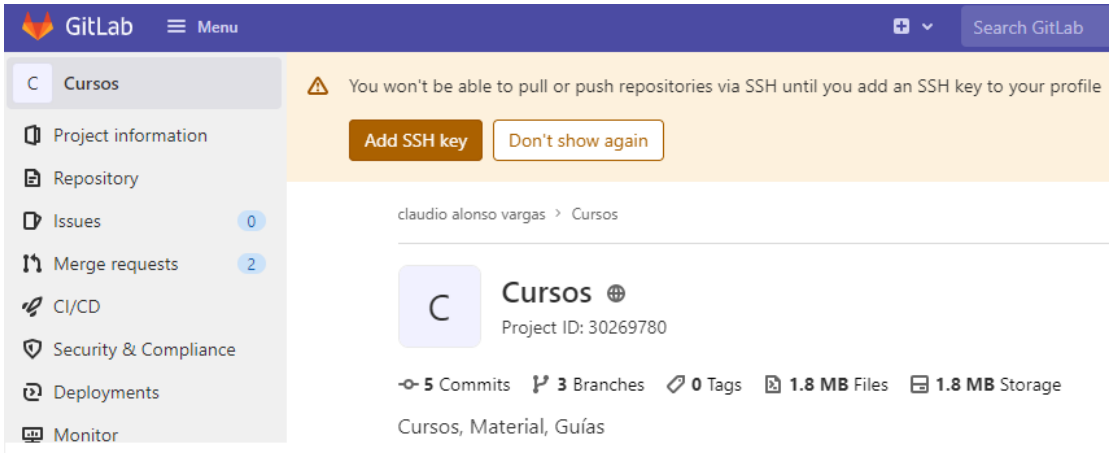
Agrega tu clave SSH pública a tu perfil de gitlab. Esto te permitirá operar en el repositorio de git desde la línea de comandos sin la necesidad de autenticación constante.

Nota: Si aún no tiene una cuenta de GitLab, el primer paso será dar de alta su usuario y contraseña.



Generar clave SSH

Iniciamos en la pestaña de SSH Key o en add SSH key en la alerta de nuestro Proyecto.



Si no tiene un par de claves SSH existente, genere uno nuevo.

1. Abra un terminal.

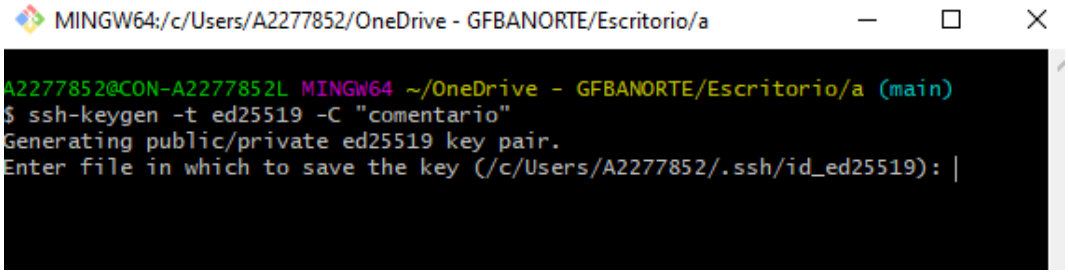
Escriba "ssh-keygen -t" seguido del tipo de clave y un comentario opcional. Este comentario se incluye en el archivo .pub que se ha creado. Es posible que desee utilizar una dirección de correo electrónico para el comentario.

Por ejemplo, para ED25519:

```
ssh-keygen -t ed25519 -C "<comment>"
```

2. Pulsa Intro.

Se muestra un resultado similar al siguiente:

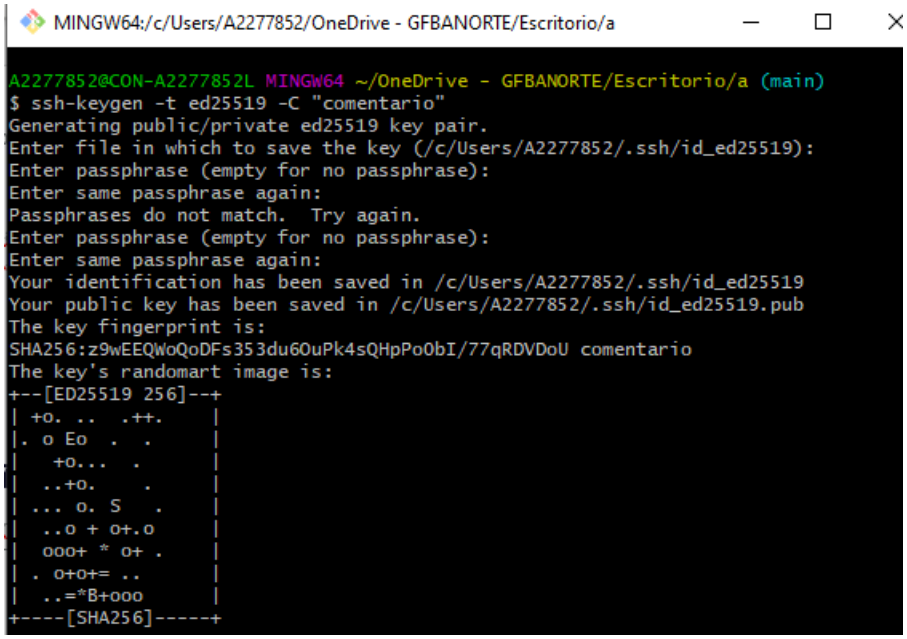


3. Se muestra una confirmación, que incluye información sobre dónde se almacenan los archivos.

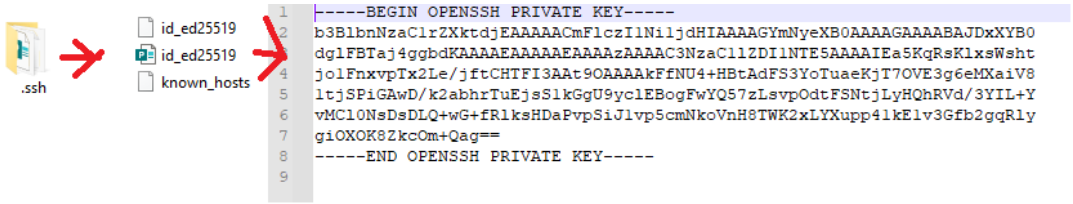
Se generan una clave pública y privada. Agregue la clave SSH Key pública a tu cuenta de GitLab y mantenga la clave privada segura.

4. Se solicitara la passphrase.

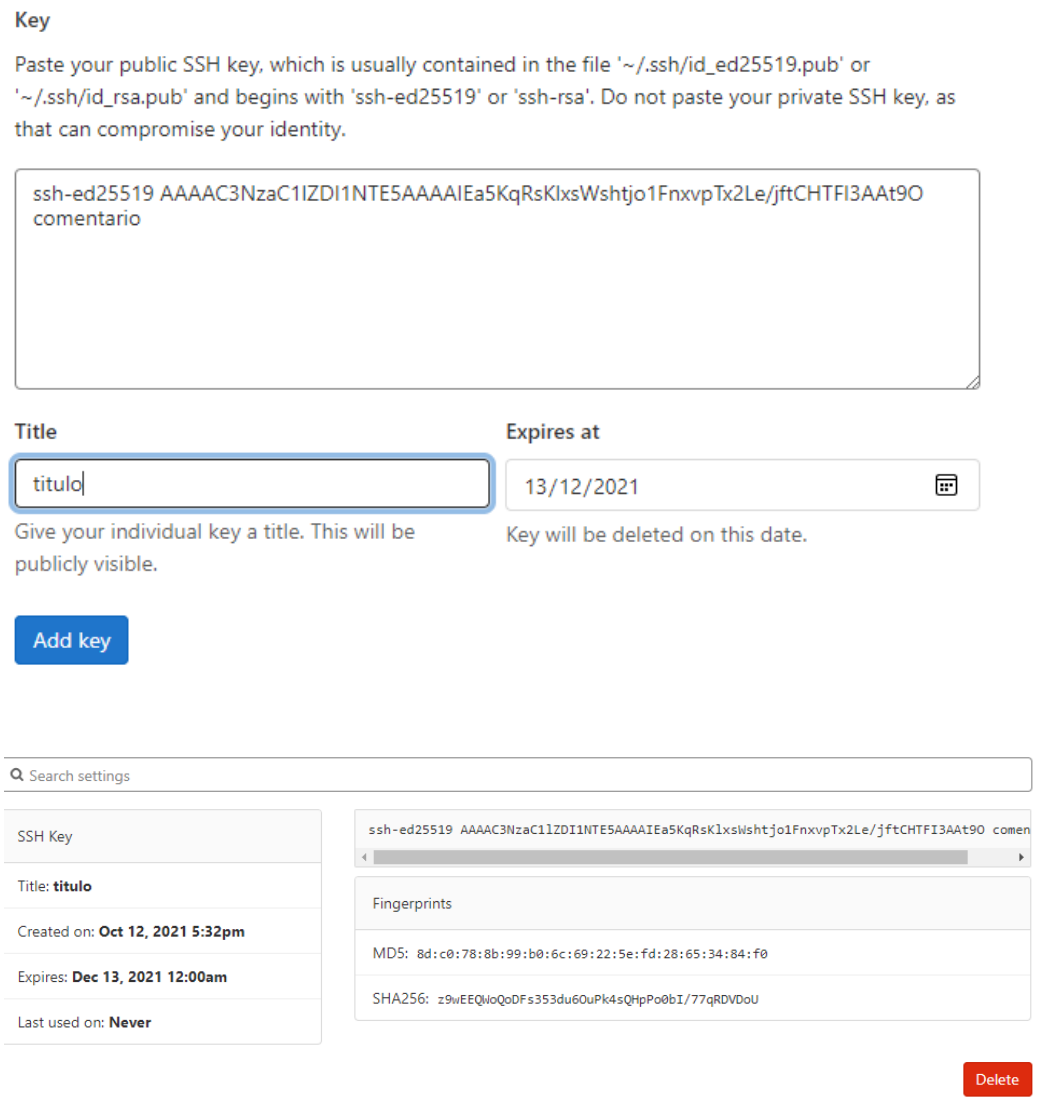
5. Finalmente se verá así nuestra SSH Key, copiamos e ingresamos en nuestro panel de GitLab.



6. Veremos en nuestro documento .ssh nuestra SSH key dentro de su documento correspondiente.



7. ¡Agregamos esta información a nuestro Proyecto GitLAb, y listo! tenemos nuestra clave SSH

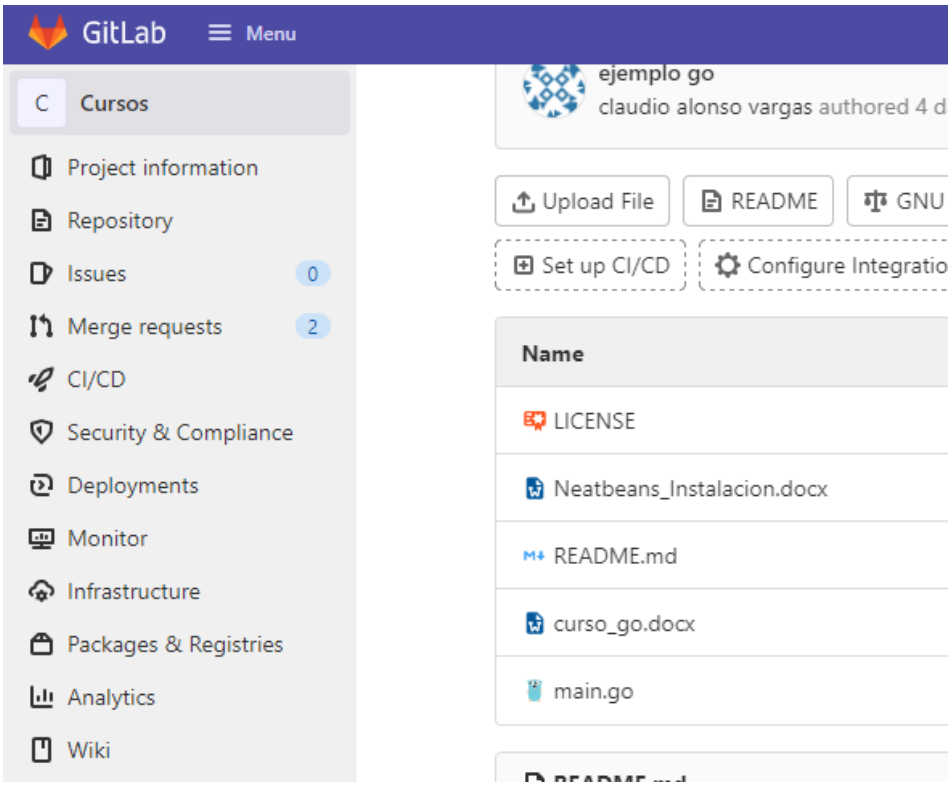


Estructura GitLab

Cada repositorio de git en gitlab está asociado a un proyecto (https:// docs.gitlab.com/ee/user/project/)

Los proyectos abarcan un repositorio de git y ofrecen herramientas adicionales a su alrededor:
Solicitudes de fusión del rastreador de problemas.
Documentación Wiki de integración continua.
…y muchos más.

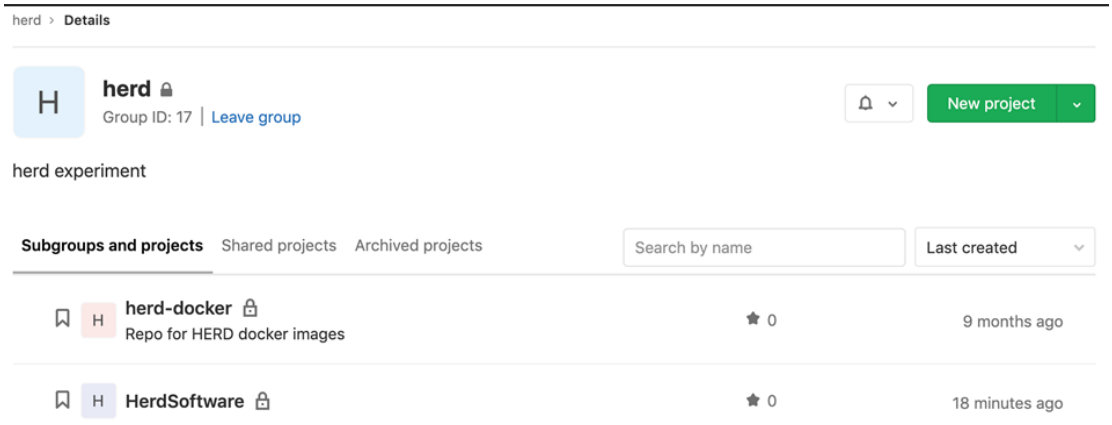
Tocaremos brevemente algunas de estas herramientas, solo para el conocimiento general, pero en principio no tendrás que lidiar directamente con la mayoría de ellas.



Se pueden juntar varios proyectos en el mismo grupo (https://docs.gitlab.com/ee/user/group/).

Los grupos se utilizan por varias razones diferentes, entre las que se encuentran:

Otorgue acceso a varios proyectos y a varios miembros del equipo en menos pasos mediante la organización de proyectos relacionados en el mismo espacio de nombres y agregando miembros al grupo de nivel superior. Haz que sea más fácil mencionar a todo tu equipo en los problemas y fusionar solicitudes creando un grupo e incluyendo a los miembros apropiados.

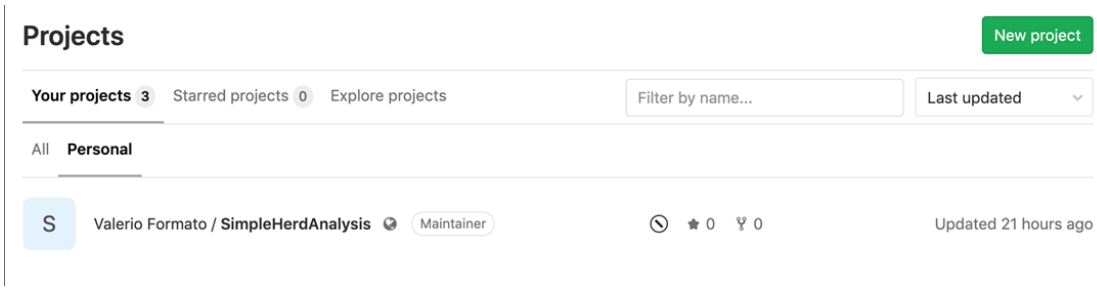


También puede crear proyectos personales, fuera del ámbito de cualquier grupo. Esto es particularmente útil si desea trabajar en su propio código de análisis personal y no necesita compartirlo con todo el grupo: ¡simplemente cree un proyecto!

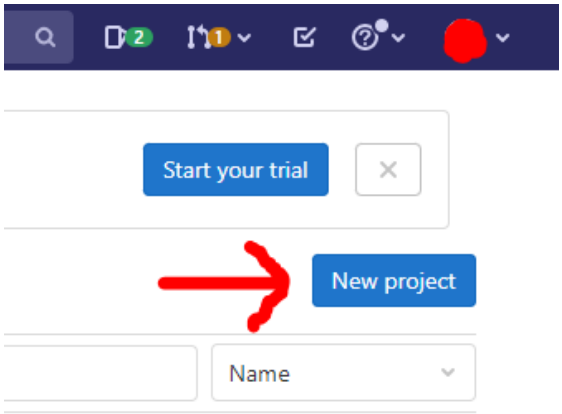
Nuevo Proyecto

Igualmente puede crear proyectos personales, fuera del ámbito de cualquier grupo. Esto es particularmente útil si desea trabajar en su propio código de análisis personal y no necesita compartirlo con todo el grupo: ¡simplemente cree un proyecto!

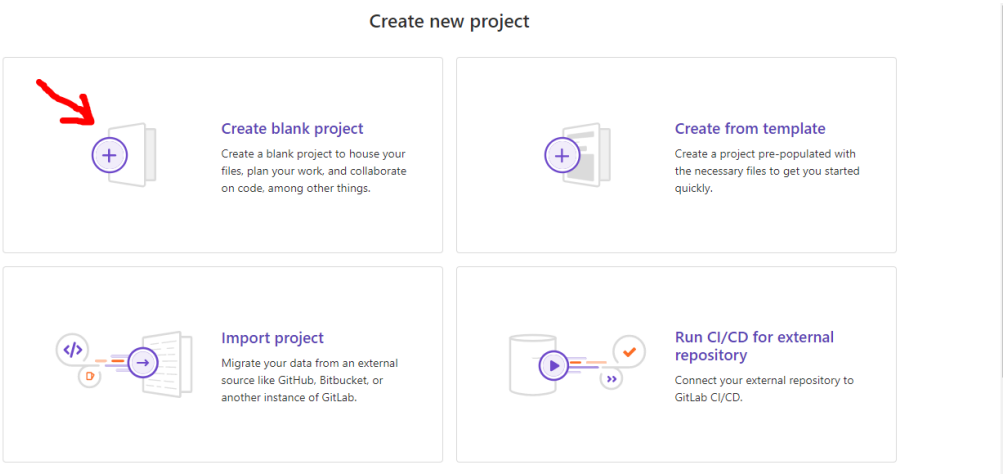
Una vez que valides tus credenciales veras la siguiente pantalla.



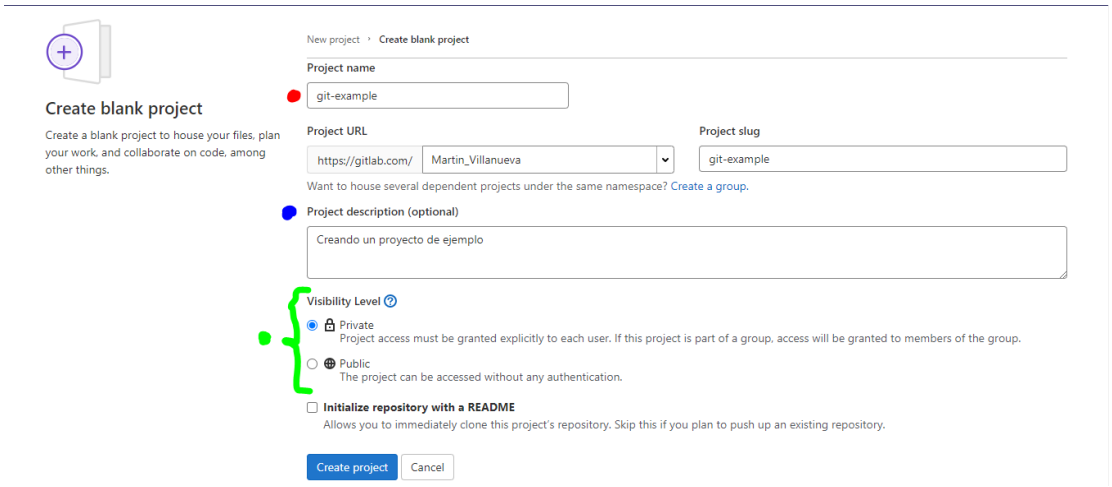
Ahora pasamos a crear un proyecto nuevo, dando clic en “New Project”, en la misma pantalla donde muestra todos tus repositorios creados:



Existen varias formas de crear un proyecto, pero en esta ocasión lo haremos en blanco, para ir viendo los comandos que nos proporciona incluso para hacer nuestros commit, desde la consola (git bash):



Ahora nos mostrara una pantalla donde vamos a llenar unos campos para la información de nuestro proyecto:



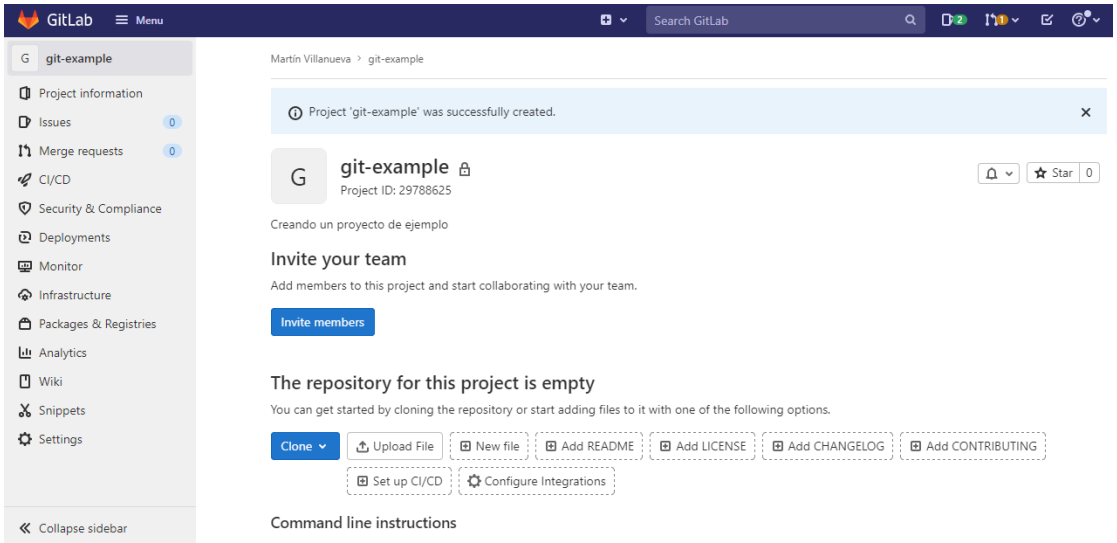
Rojo: Es el nombre de tu proyecto.

Azul: Puedes dar una descripción de tu proyecto.

Verde: Son los niveles de permisos para los usuarios que en este caso sería multiplayer o público.

Una buena práctica es agregar un documento README con descripción del proyecto, notas del autor o autores.

Ahora daremos clic en “Create Project”, lo cual generará nuestro proyecto git-lab:



En la parte de abajo veremos los comandos para ejecutar en git bash:

Clonar un repositorio

Cree un proyecto en su espacio de “nombres de usuario”.

Nota: Si no sabes agregar tu nombre de usuario Git, verifica este paso en la guía de git.

Elija importar desde cualquier repositorio por URL ejemplo::

https://gitlab.com/gitlab-org/training-examples.git

¡Cree un directorio “workspace” en tu directorio de inicio seguido de copiar y ejecutar la siguiente línea y listo! tienes una copia en tu repositorio git.

```
mkdir ~/workspace
cd ~/workspace

git clone git@gitlab.example.com:<username>/training-examples.git
cd training-examples
```

Si hemos configurado nuestra clave SSH se clonará sin problemas.

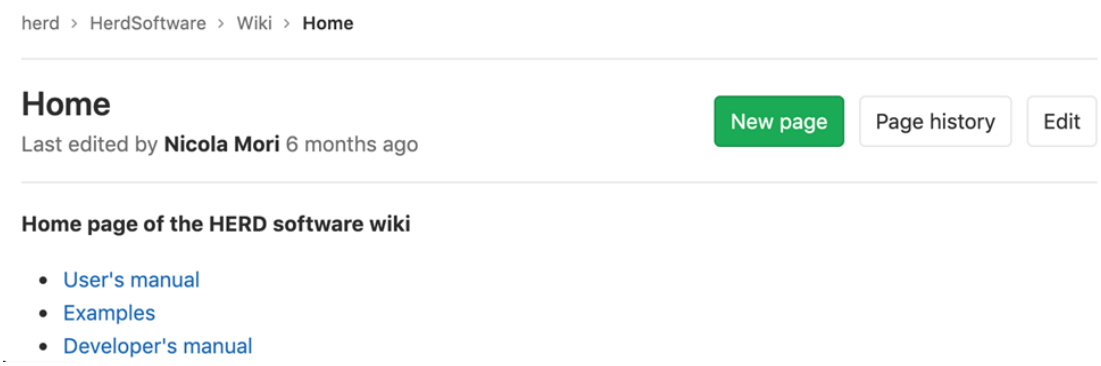
```
gme@ubuntu:~$ mkdir mydir
gme@ubuntu:~$ cd mydir
gme@ubuntu:~/mydir$ git clone http://gitlab.mcnz.com/root/gi
tlab-made-easy.git
Cloning into 'gitlab-made-easy'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 20 (delta 7), reused 10 (delta 2), pack-reused
 0
Unpacking objects: 100% (20/20), 2.07 KiB | 706.00 KiB/s, do
ne.
gme@ubuntu:~/mydir$ ^C
gme@ubuntu:~/mydir$ cd gitlab-made-easy
gme@ubuntu:~/mydir/gitlab-made-easy$ ls
feature-file.html  my-hotfix.html  README.md  style.css
home.html          phd.html        script.js
gme@ubuntu:~/mydir/gitlab-made-easy$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
gme@ubuntu:~/mydir/gitlab-made-easy$ _
```

Wiki

Un sistema separado para la documentación llamado Wiki, está integrado en cada proyecto de GitLab. Está habilitado de forma predeterminada en todos los proyectos nuevos y puede encontrarlo en Wiki en su proyecto.

Los wikis son muy convenientes si no desea mantener su documentación en su repositorio, pero sí desea mantenerla en el mismo proyecto donde reside su código.

Puede crear páginas Wiki en la interfaz web o localmente usando git ya que cada Wiki es un repositorio git separado.



Un vistazo a Wiki

Las wikis son una clase de biblioteca para nuestros repositorios, algo así como una guía de lo que se está haciendo.

Configure una rama predeterminada para tu wiki, introducido en GitLab 14.1.

La rama predeterminada para su repositorio wiki depende de tu versión de GitLab:

- Versiones 14.1 y posteriores de GitLab: las wikis heredan el nombre de rama predeterminado configurado para su instancia o grupo. Si no se configura ningún valor personalizado, GitLab usa main.
- Versiones de GitLab 14.0 y anteriores: GitLab usa master.

Para cualquier versión de GitLab, puede cambiar el nombre de esta rama predeterminada para wikis creados previamente.

Crea la página de inicio de la wiki

Cuando se crea una wiki, está vacía. En su primera visita, puede crear la página de inicio que los usuarios ven cuando ven el wiki. Esta página requiere un título específico para usarse como página de inicio de su wiki. Para crearlo:

- Vaya a su proyecto o grupo y seleccione Wiki.
- Seleccione, Crear su primera página.
- GitLab requiere que esta primera página se titule home. La página con este título sirve como la página principal de su wiki.

- Seleccione un formato para diseñar su texto.
- Agregue un mensaje de bienvenida para su página de inicio en la sección Contenido. Siempre puedes editarlo más tarde.
- Agrega un mensaje de confirmación. Git requiere un mensaje de confirmación, por lo que GitLab crea uno si no ingresa uno usted mismo.
- Seleccione, Crear página.

Crea una nueva página wiki

Los usuarios con el rol de desarrollador pueden crear nuevas páginas wiki:

- Vaya a su proyecto o grupo y seleccione Wiki.
- Seleccione, Nueva página en esta página o en cualquier otra página wiki.
- Seleccione un formato de contenido.
- Agrega un título para tu nueva página. Los títulos de las páginas utilizan caracteres especiales para subdirectorios y formato, y tienen restricciones de longitud.
- Agrega contenido a tu página wiki.
- (Opcional) Adjunte un archivo y GitLab lo almacenará de acuerdo con su versión instalada de GitLab:
- Archivos agregados en GitLab 11.3 y posterior: los archivos se almacenan en el repositorio de Git de la wiki.
- Archivos agregados GitLab 11.2 y anteriores: los archivos se almacenan en el propio GitLab. Para agregar el archivo al repositorio Git de la wiki, debe volver a cargar el archivo.
- Agrega un mensaje de confirmación. Git requiere un mensaje de confirmación, por lo que GitLab crea uno si no ingresa uno usted mismo.
- Seleccione, Crear página.

Editar una página wiki

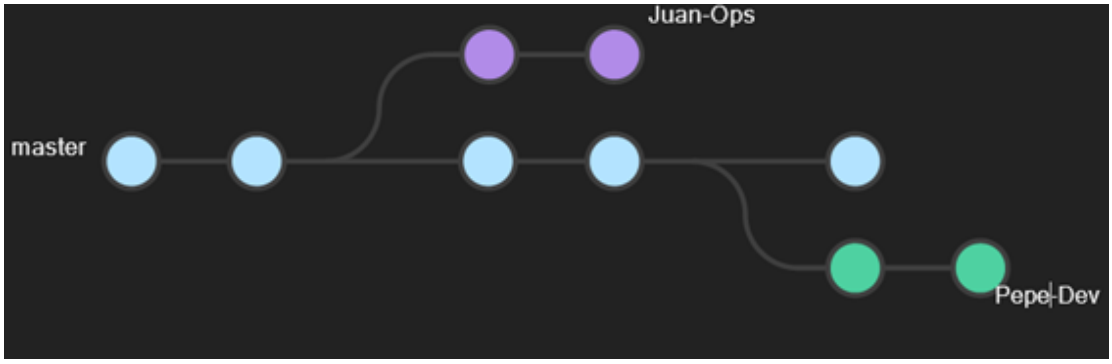
Necesita al menos el rol de Desarrollador para editar una página wiki:

- Ve a tu proyecto o grupo y seleccione Wiki.
- Ve a la página que desea editar.
- Seleccione el icono de edición ().
- Edita el contenido.
- Seleccione, Guardar cambios.

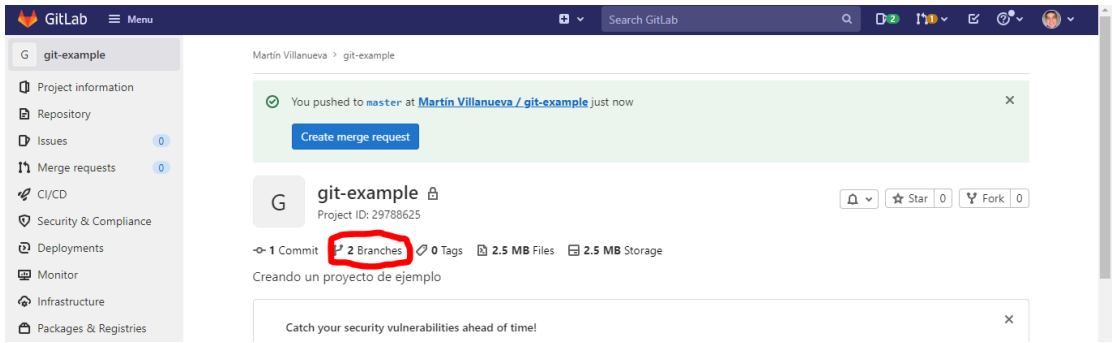
Trabajando con Ramas

Esperamos que al principio el trabajo consista tanto en el análisis de datos de los archivos simulados (que se pueden hacer en proyectos personales), como en la mejora y desarrollo de nuevo código de reconstrucción.

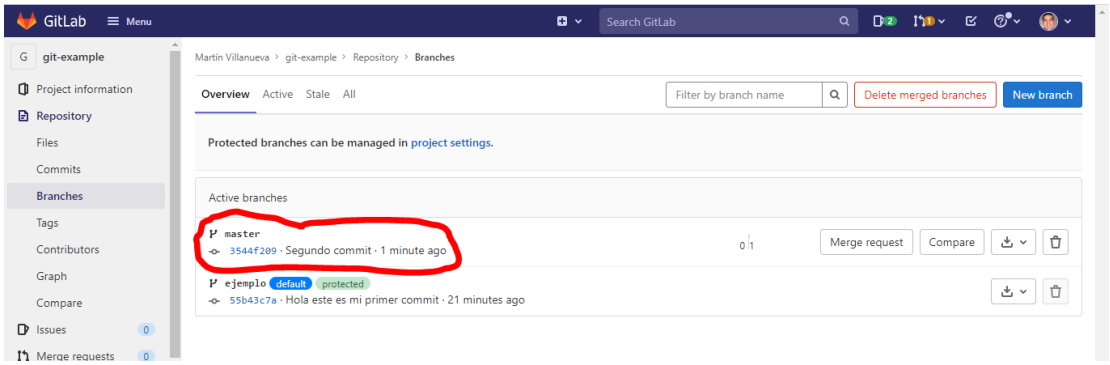
La rama predeterminada es la rama principal. Para permitir que todos trabajen libremente y mantener el código en un buen estado limpio y funcional, es preferible desarrollar nuevas funciones o correcciones en ramas dedicadas y fusionarlas en master solo cuando el trabajo esté terminado.



Ahora veamos un Branch actual en nuestro repositorio en git-lab, este segundo Branch se hizo desde el git bash de nuestra máquina local, ahora da clic en esas ramas:



Verás que puedes crear una rama en el botón de New Branch, junto al cual podemos solicitar un merge para unir nuestros cambios a la rama que seleccionemos.



Integración Continua (CI)

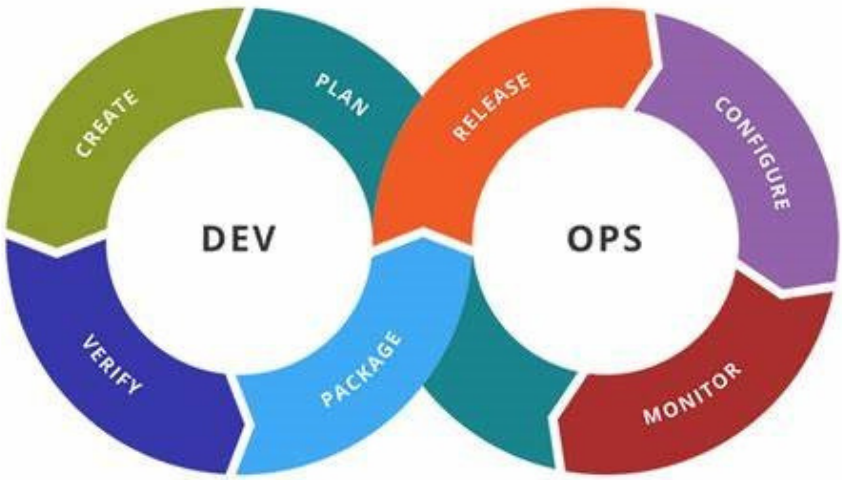
Continuous Integration (CI) es la práctica de integrar código en un repositorio compartido y construir / probar cada cambio automáticamente, tan pronto como sea posible, generalmente varias veces al día.

Prácticamente, esto se traduce en un conjunto de comprobaciones y pruebas que se realizarán automáticamente en cada inserción y alertarán al desarrollador si alguna de estas comprobaciones falla.

Tanto el proceso de construcción como las pruebas unitarias se verifican usando el CI, en 4 entornos posibles (centos7, ubuntu18, ubuntu16 y OSX High Sierra)

Más Información

GitLab Continuous Integration (CI)



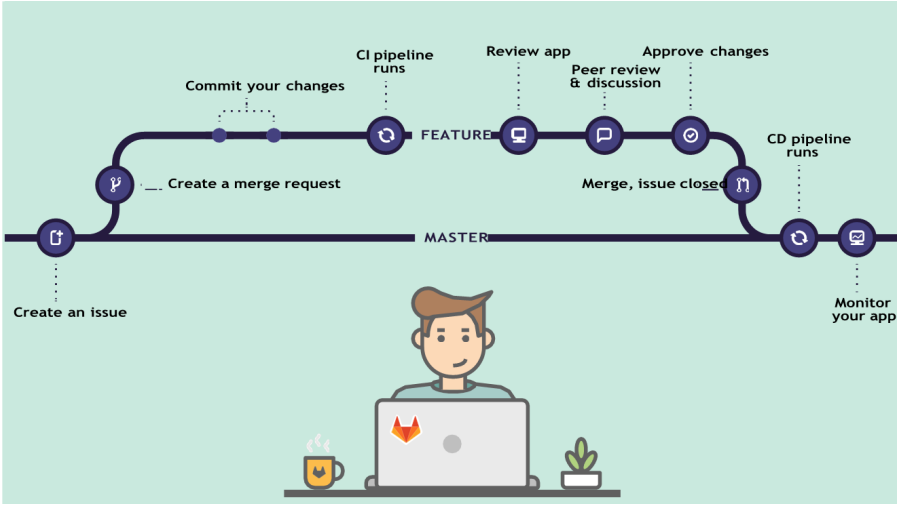
Push un nuevo Merge, Commit y Pipeline

MERGE REQUEST

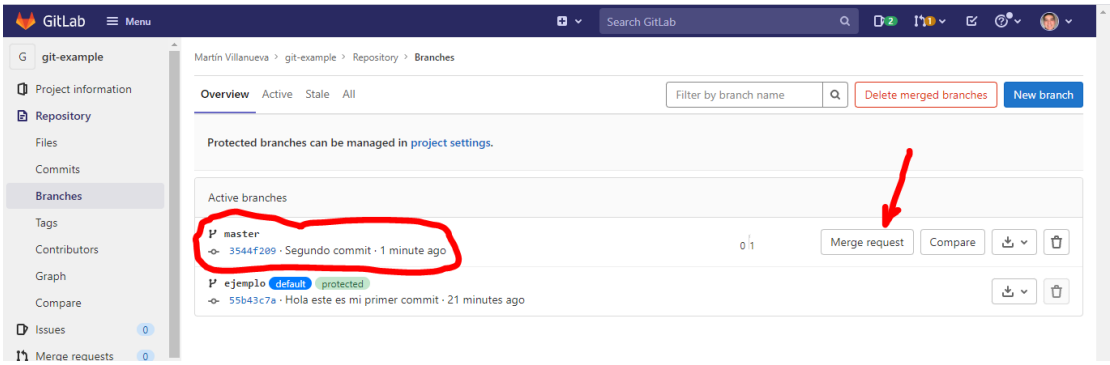
Un Merge Request (MR) es la base de GitLab como plataforma de control de versiones y colaboración de código.

Es tan simple como su nombre lo indica: una solicitud para fusionar una rama con otra.

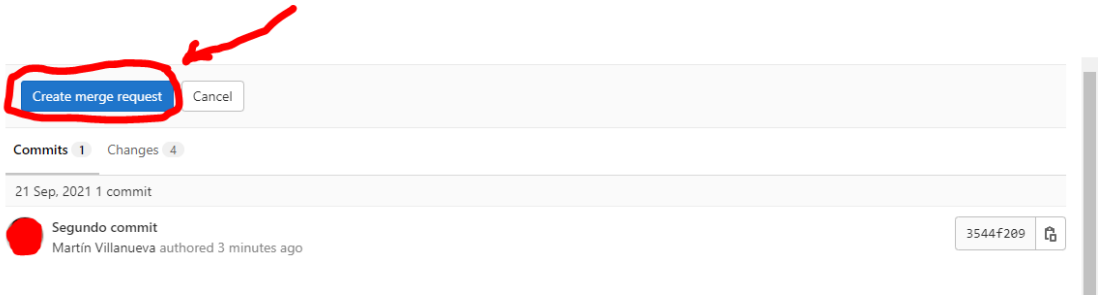
- Compara los cambios entre dos ramas.
- Revisar y discutir las modificaciones propuestas en línea.
- Cree, pruebe e implemente su código por rama con GitLab CI / CD integrado.
- Cerrar automáticamente los problemas que originaron la implementación propuesta en la solicitud de fusión.
- Organice sus problemas y combine solicitudes de manera coherente a lo largo del Proyecto con etiquetas.
- Resolver conflictos de combinación desde la interfaz de usuario
- Habilitar solicitudes de combinación de avance rápido.



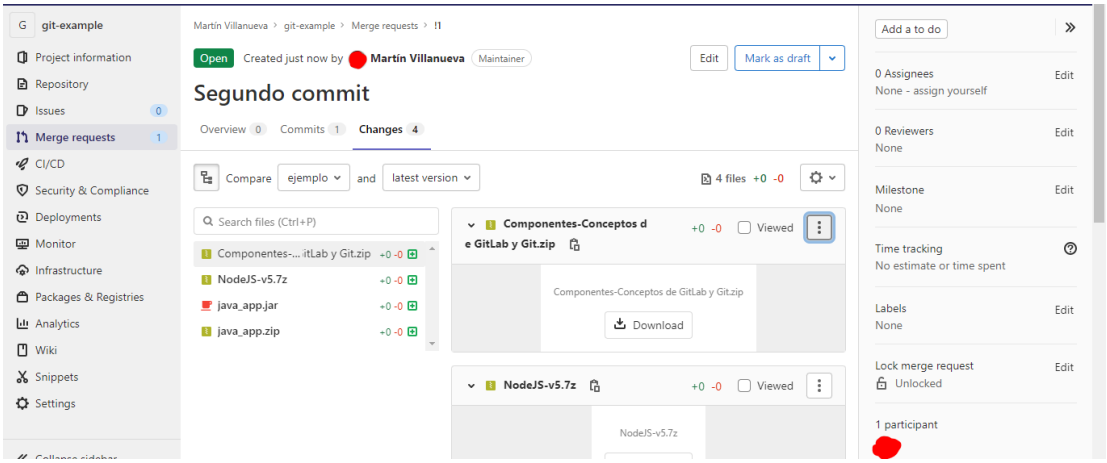
Iniciamos solicitando un merge en el botón de merge request:



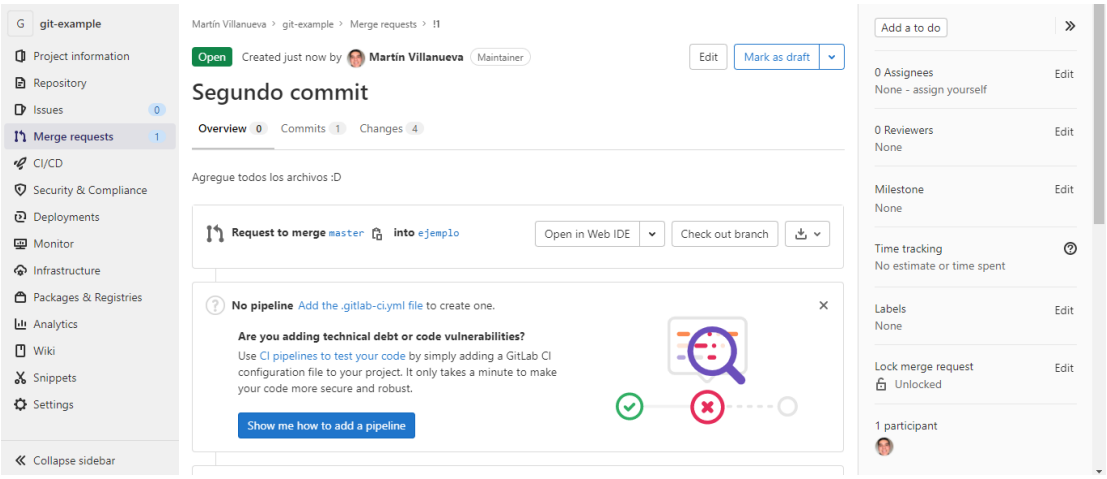
Ahora nos mostrara una pantalla para ello daremos clic en "Create merge request", aquí es para hacer la fusión de nuestra rama que viene de git local hacia la plataforma git-lab:



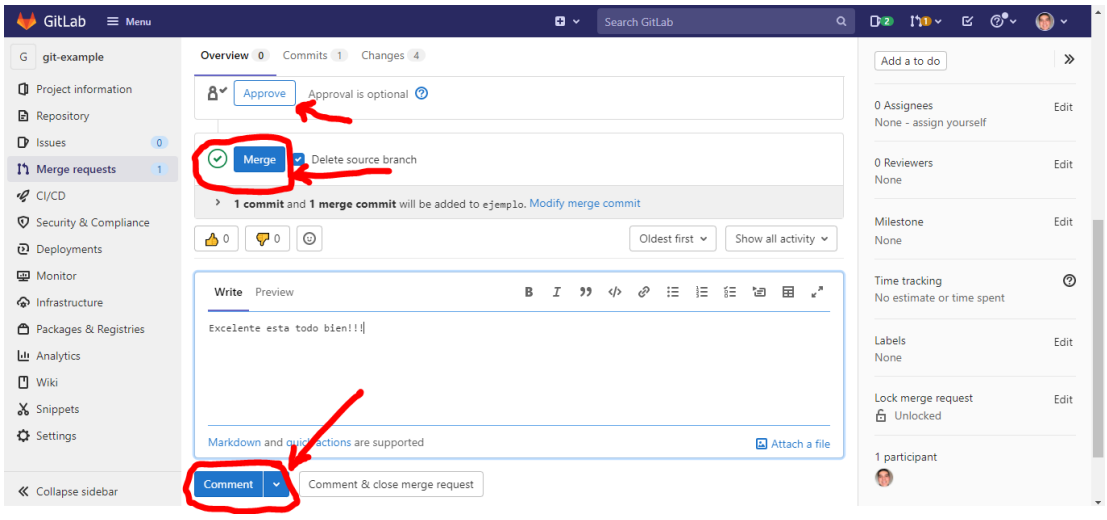
Al dar clic nos mostrara esta pantalla, es un estado de nuestro commit y otras informaciones que podrás observar:



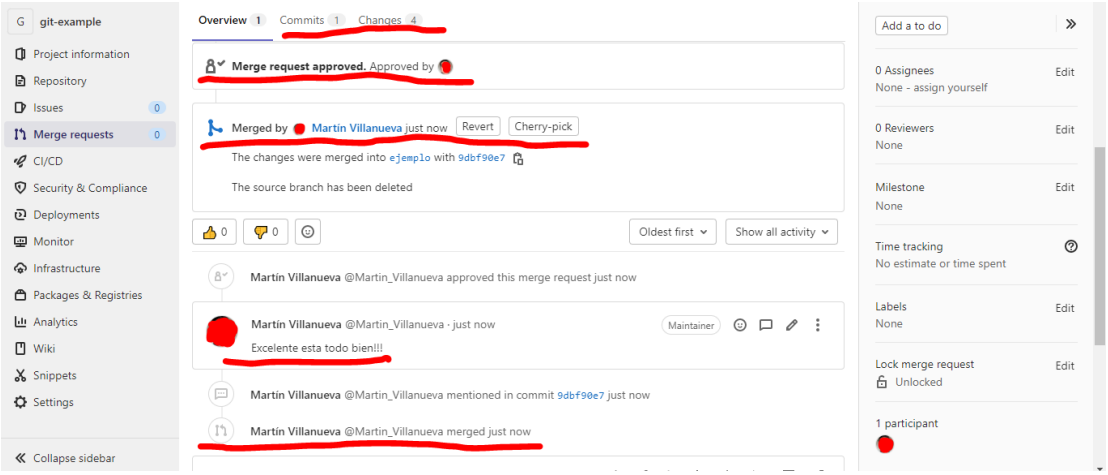
Aquí vemos como nuestra petición fue fusionada de máster a ejemplo el nivel de las ramas:



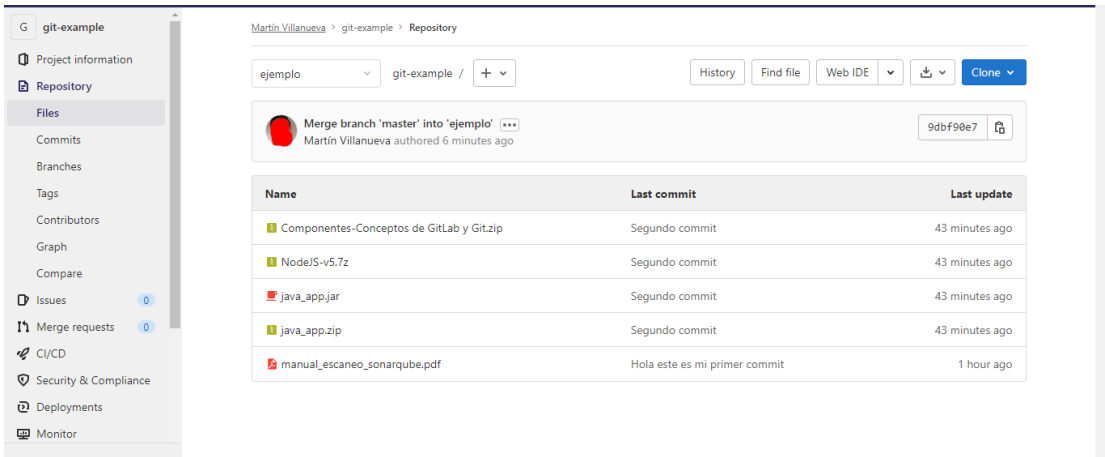
Mas abajo veremos como viene un botón de aprobar, hacer fusión y comentar, puedes comentar lo que gustes de acuerdo con el commit que hiciste, con los archivos que añadiste, modificaste, etc.



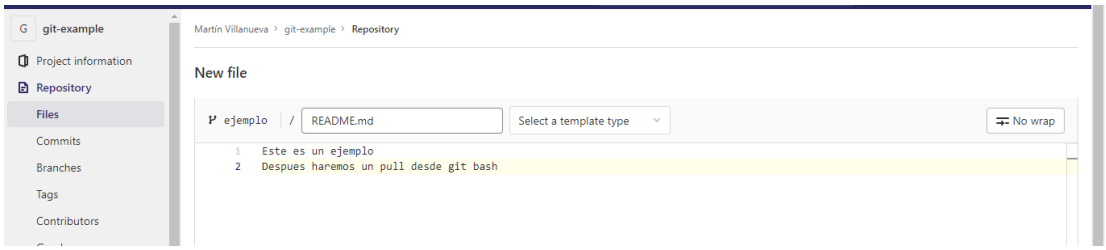
Ahora veamos cómo se ve haciendo el cambio:

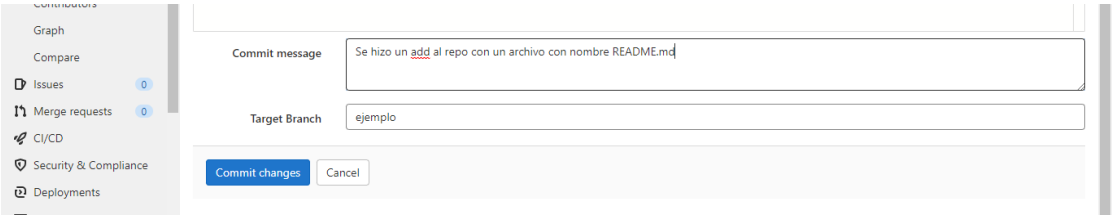


A continuación los cambios reflejados ya en nuestro repositorio:

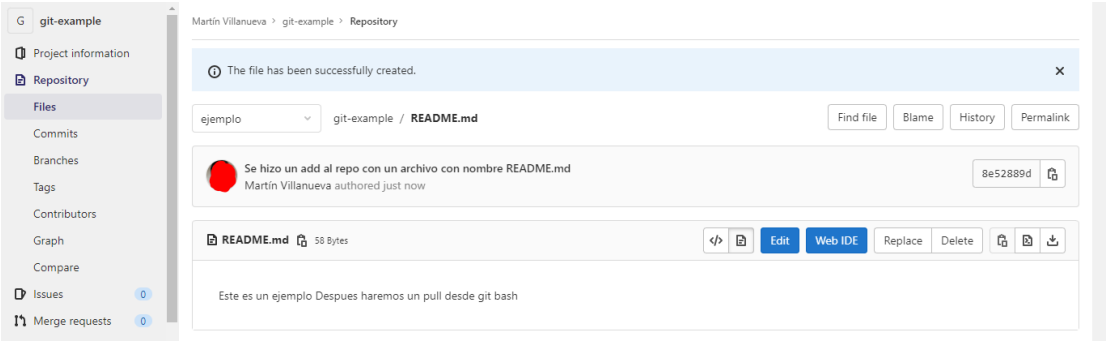


Ahora vamos a hacer un “Commit changes” desde git-lab:





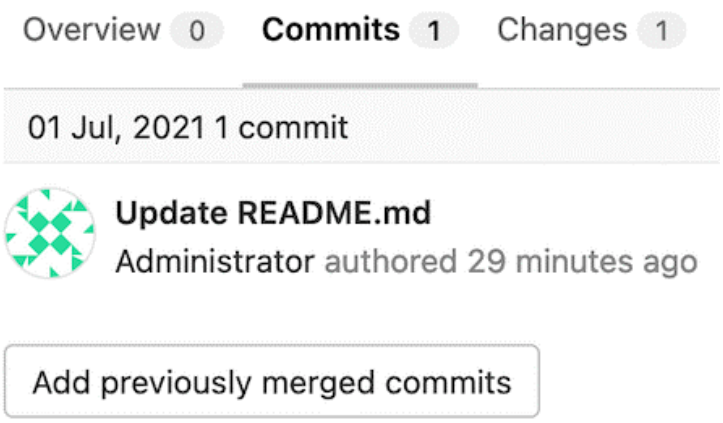
Ahora veremos los cambios:



COMMIT

Para agregar confirmaciones combinadas previamente a una solicitud de combinación para obtener más contexto:

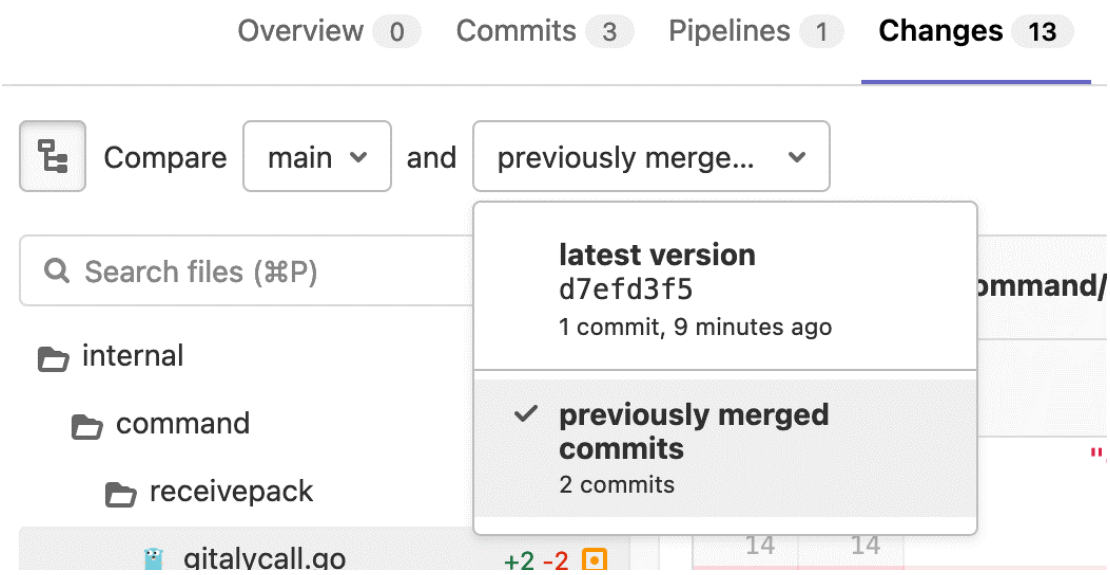
- Vaya a su solicitud de fusión (merge request).
- Seleccione la pestaña Confirmaciones (Commits).
- Desplácese hasta el final de la lista de confirmaciones y seleccione Add previously previamente combinadas:



- Seleccione las confirmaciones que desea agregar.
- Seleccione Guardar cambios.

Para ver los cambios realizados en esas confirmaciones combinadas anteriormente:

En la solicitud de combinación, seleccione la pestaña Cambios. Desplácese hasta (árbol de archivos) Compare y seleccione confirmaciones combinadas anteriormente:

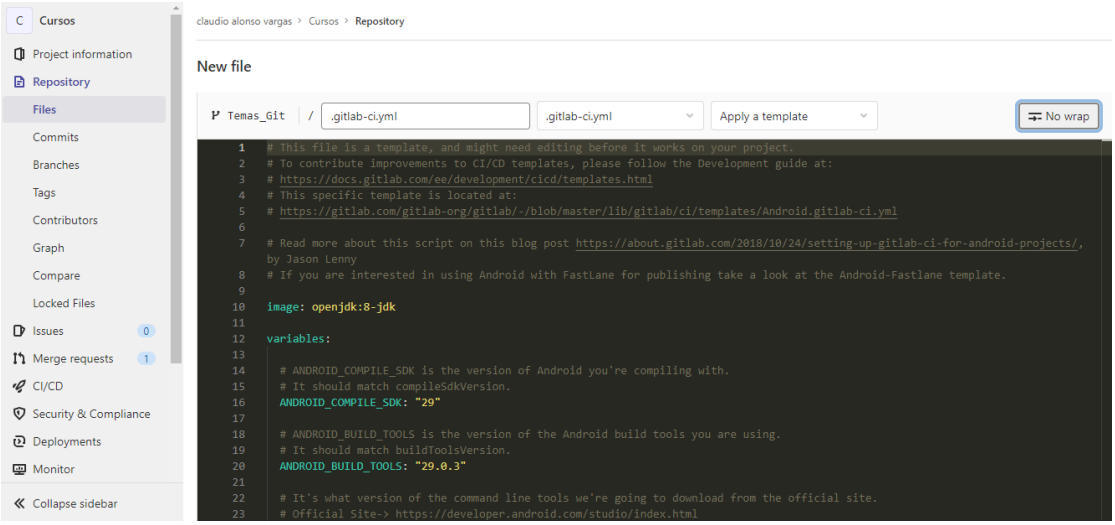


PIPELINE

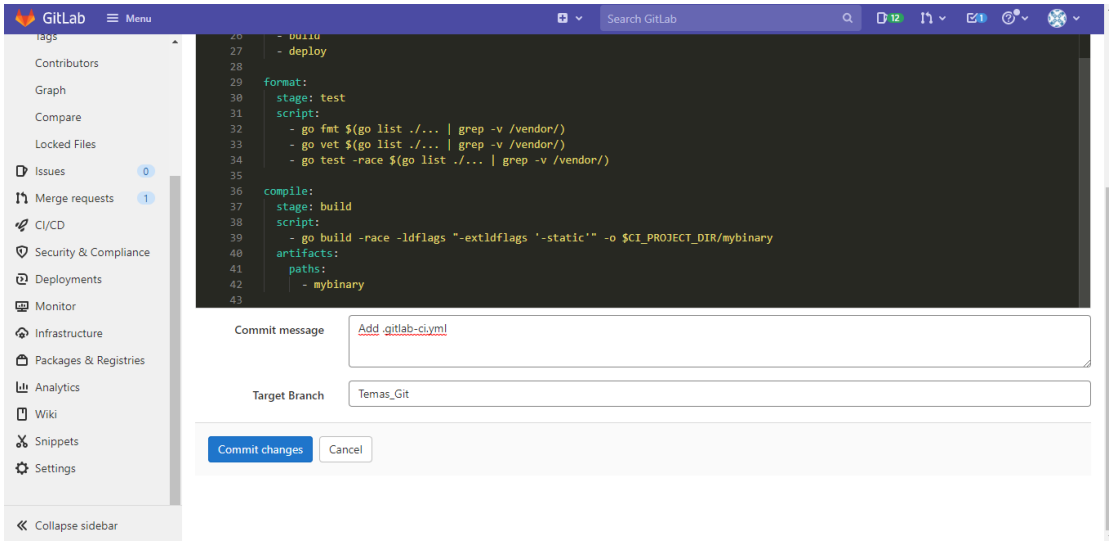
Un pipeline es la manera abreviada de referirse a dicho cúmulo de etapas en un proyecto. El ejemplo que vamos a plantear será muy simple en cuanto a funcionalidad.

Estamos agregando un archivo de configuración de GitLab CI para agregar un pipeline al proyecto. Puede crearlo manualmente, pero le recomendamos que comience con una plantilla de GitLab que funcione de inmediato.

Elija Calidad de código para agregar un pipeline que prueba la calidad del código.

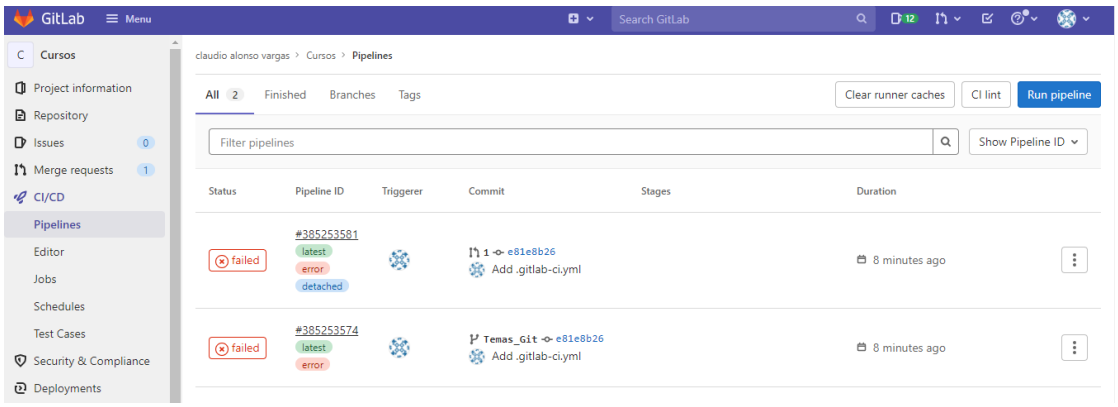


Cuando la plantilla esta lista eligiendo en la rama, nombre, y plantilla ya podemos hacer un commit para nuestro primer pipeline.



El pipeline probará el código en cada confirmación.

Aparecerá un informe de calidad de código en las solicitudes de combinación para advertirle sobre posibles degradaciones del código.



Herramientas Avanzadas

Los issues son el medio fundamental para colaborar en ideas y planificar el trabajo en GitLab.

Se utilizan para informar un error, proponer una nueva característica o elemento de trabajo, organizar y dividir el trabajo en grandes tareas entre diferentes desarrolladores, puede comprobar / agregar nuevos problemas desde la página de la lista de problemas...cada número viene con un título y una descripción. Se pueden agregar comentarios a medida que avanza el trabajo. Gitlab puede crear automáticamente una rama dedicada (y una solicitud de fusión) para un problema determinado. Los problemas se pueden etiquetar con etiquetas personalizadas.

Resumen

INSTALACIÓN

Lo primero que tienes que hacer es crear una cuenta en GitLab desde su sitio: https://gitlab.com/users/sign_in

Después necesitas Git, puedes bajarlo desde la página oficial e instalarlo como cualquier otro programa:
https://git-scm.com

CONFIGURACIÓN GIT

Una vez instalado Git abre la terminal y configura la información del usuario utilizada en todos los repositorios locales.

Establece un nombre identificable y la dirección de correo electrónico que se adjuntara a sus confirmaciones (commits) y etiquetas (tags):

```
git config --global user.name “[firstname lastname]”
git config --global user.email “[valid_email]”
```

INICIAR UN PROYECTO

Configura la información, inicializando o clonando el repositorio.

Inicializa un directorio existente como un repositorio de Git:

```
git init
```

Si no se proporciona el nombre del proyecto, Git lo puede crear con:

```
git init [project_name]
```

O también se puede descargar un proyecto (fork) con el historial completo del repositorio remoto:

```
git clone [project_url]
```

TRABAJO DIARIO

Muestra el estado de su directorio de trabajo, preparados para su próxima confirmación (commit):

```
git status
```

Agregue un archivo al área de preparación (staging) para su próxima confirmación (commit):

```
git add [file]
```

Agregue todos los archivos pendientes al área de preparación (staging) para su próxima confirmación (commit):

```
git add .
```

Revierte tu repositorio a un estado de trabajo anterior:

```
git reset [file]
```

Descarta cambios en el directorio de trabajo. Esta operación es irrecuperable:

```
git checkout -- [file]
```

Muestra cambios entre el directorio de trabajo y el área de preparación (staging):

```
git diff [file]
```

Muestra cualquier cambio entre el área de preparación (staging) y el repositorio:

```
git diff --staged [file]
```

Crea una nueva confirmación (commit) a partir de los cambios agregados al área de preparación (staging):

```
git commit -m “[descriptive_message]”
```

MODELO DE RAMIFICACIÓN

Enlista todas las ramas locales en el repositorio. Con
-a: muestra todas las ramas:

```
git branch [-a]
```

Crea una nueva rama (branch), haciendo referencia a la cabecera actual HEAD:

```
git branch [branch_name]
```

Cambia el directorio de trabajo a la rama (branch) especificada. Con -b: Git creará la rama especificada, si no existe:

```
git checkout [-b][branch_name]
```

Fusiona el historial de la rama (branch) especificada en el actual:

```
git merge [branch_name]
```

Elimina la rama (branch) seleccionada, si ya está fusionada con cualquier otra. -D en lugar de -d fuerza la eliminación:

```
git branch -d [branch_name]
```

REVISA TU TRABAJO

Muestra el historial de confirmaciones de la rama actual. -n count muestra las últimas n confirmaciones (commits):

```
git log [-n count]
```

Muestra una descripción general con etiquetas de referencia y gráfico histórico:

```
git log --oneline --graph --decorate
```

Enlista las confirmaciones (commits) que están presentes en la rama (branch) actual y no se fusionaron en la ref. Una ref.. puede ser un nombre de rama (branch) o un nombre de etiqueta (tag):

```
git log ref..
```

Enlista las confirmaciones (commits) que están presentes en la referencia y no se fusionaron con la rama (branch) actual:

```
git log ..ref
```

Lista de operaciones (por ejemplo, checkouts o commits) realizadas en el repositorio local:

```
git reflog
```

ETIQUETADO

Generalmente, el etiquetado se usa para capturar un punto en el historial que se utiliza para una publicación de versión (por ejemplo, v1.0.1).

Enlista todas las etiquetas:

```
git tag
```

Crea una etiqueta para una confirmación específica, con el nombre de la etiqueta y el SHA de confirmación (commit_sha):

```
git tag [tag_name] [commit_sha]
```

Las etiquetas anotadas se almacenan como objetos completos en la base de datos de Git, es decir, almacena metadatos adicionales como: nombre de la persona que etiqueta, su correo electrónico y la fecha. Al igual que las confirmaciones (commits) y mensajes de las confirmaciones.

Crea un objeto de etiqueta llamado [name] para la confirmación actual:

```
git tag -a [tag_name] [commit_sha]
```

Crea un objeto de etiqueta con un mensaje:

```
git tag -a [tag_name] -m “[descriptive_message]”
```

Eliminar una etiqueta del repositorio local:

```
git tag -d [tag_name]
```

REVERTIR CAMBIOS

Cuando se trabaja en un proyecto de equipo, es bastante común que los desarrolladores creen bifurcaciones, agreguen archivos y los organicen para confirmaciones (commits) cuando esten listos. Sin embargo, en algunos casos, es posible que se de cuenta de que los cambios que realizó no fueron tan buenos.

Limpia el área de preparación (stage), reescribe el árbol de trabajo de la confirmación especificada (commit):

```
git reset --hard [commit]
```

Crea una nueva confirmación (commit), revirtiendo los cambios de la confirmación especificada. Genera una inversión de cambios:

```
git revert [commit sha]
```

SINCRONIZAR REPOSITORIOS

En muchas ocasiones se requiere descargar confirmaciones, archivos y referencias de un repositorio remoto en el repositorio local para visualizar en que han estado trabajando otros desarrolladores.

Obtiene todas las ramas y cambios del repositorio. Pero no actualiza las ramas de seguimiento:

```
git fetch [remote]
```

Extrae los cambios de la rama remota e integralos con la rama local actual:

```
git pull [remote]
```

Empuje los cambios locales al repositorio remoto. Utilice --tags para insertar etiquetas:

```
git push [--tags] [remote]
```

Inserta una rama local en otro repositorio, especifica el nombre remoto correcto, así como la rama que se va a insertar:

```
git push -u [remote] [branch]
```

SEGUIMIENTO Y CAMBIO DE RUTA

Elimina el archivo del directorio de trabajo y del área de preparación:

```
git rm [file]
```

Cambiar una ruta de archivo existente:

```
git mv [existing-path] [new-path]
```

Muestra todos los registros de confirmación (commits) con indicación de las rutas que se movieron:

```
git log --stat -M
```

COMMITS TEMPORALES

Guarda todos los cambios actuales en el directorio de trabajo para su uso posterior:

```
git stash
```

Enlista la pila de cambios de archivos guardados:

```
git stash list
```

Aplica el contenido del área de trabajo almacenado y limpia el área de trabajo:

```
git stash pop
```

Elimina los cambios de la parte superior de la pila de archivos guardados:

```
git stash drop
```

Explorador de comandos de Git

https://gitexplorer.com

IGNORAR ARCHIVOS

Evita la puesta en escena o la confirmación involuntaria de archivos.

Verifique que el archivo .gitignore exista en el proyecto e ignore cierto tipo de archivos:

```
/logs/*
!logs/.gitkeep
/tmp
*.swp
```

DOCUMENTACIÓN

Comenzar a usar Git en la línea de comandos:

Get Started

Más Información

Departamento de Arquitectura TI

Autores de publicación:

Claudio Alonso Vargas
claudio.vargas.organista@banorte.com

Martin Mesinos Villanueva
martin.mesinos.villanueva@banorte.com

Alexis Jesus Rodriguez
alexis.arrambide.rodriguez@banorte.com